

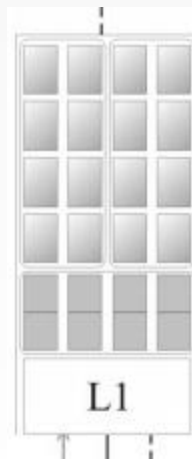


NVidia CUDA

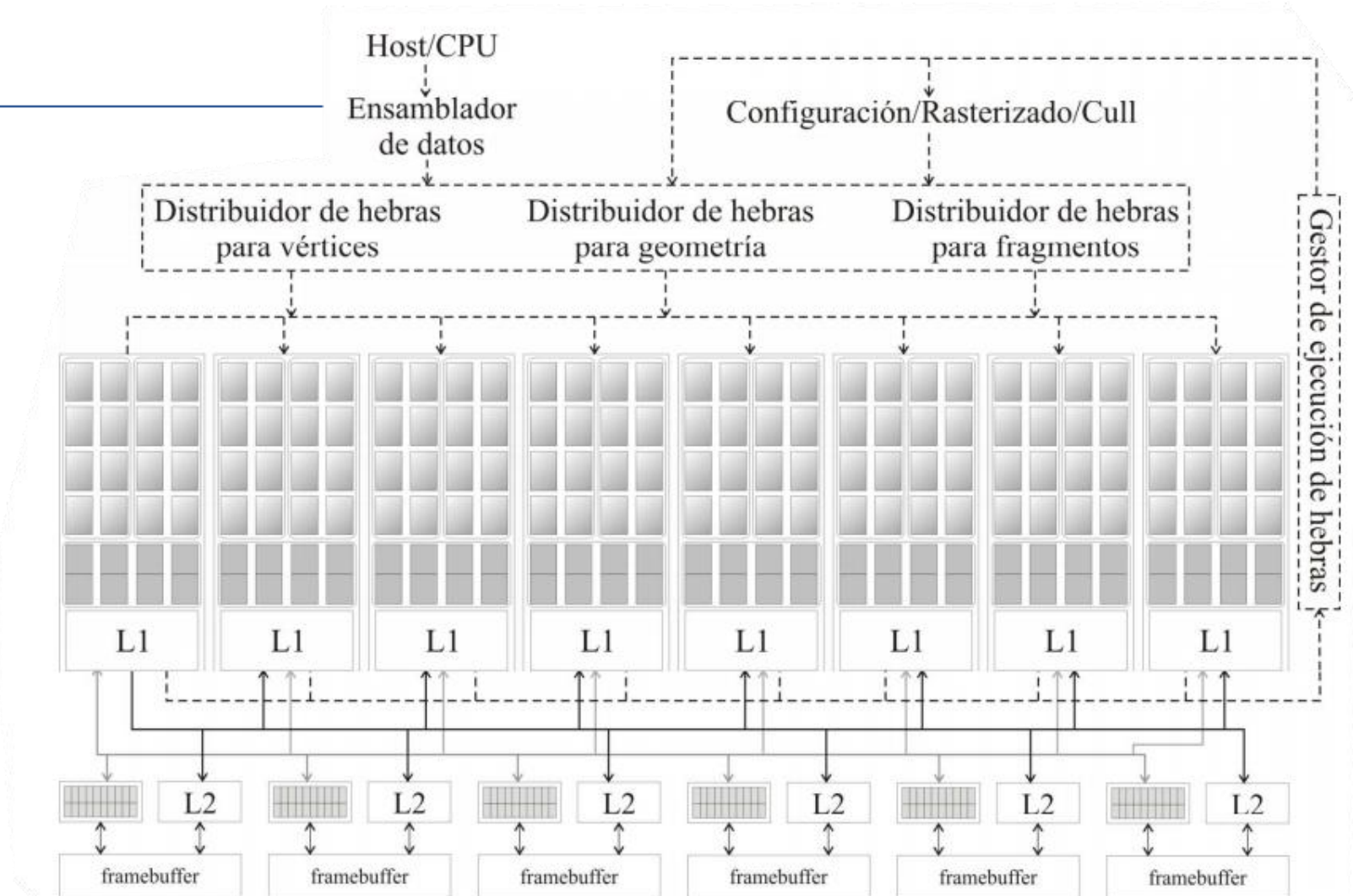
Marcos.novalbos@u-tad.com

NVIDIA Architecture

- Streaming multiprocessor:
 - Multiprocessor blocks
 - Each streaming processor contains:
 - An L1 cache shared with the processors in the same block
 - A “fast” memory area shared by the block
 - A communication bus to main memory

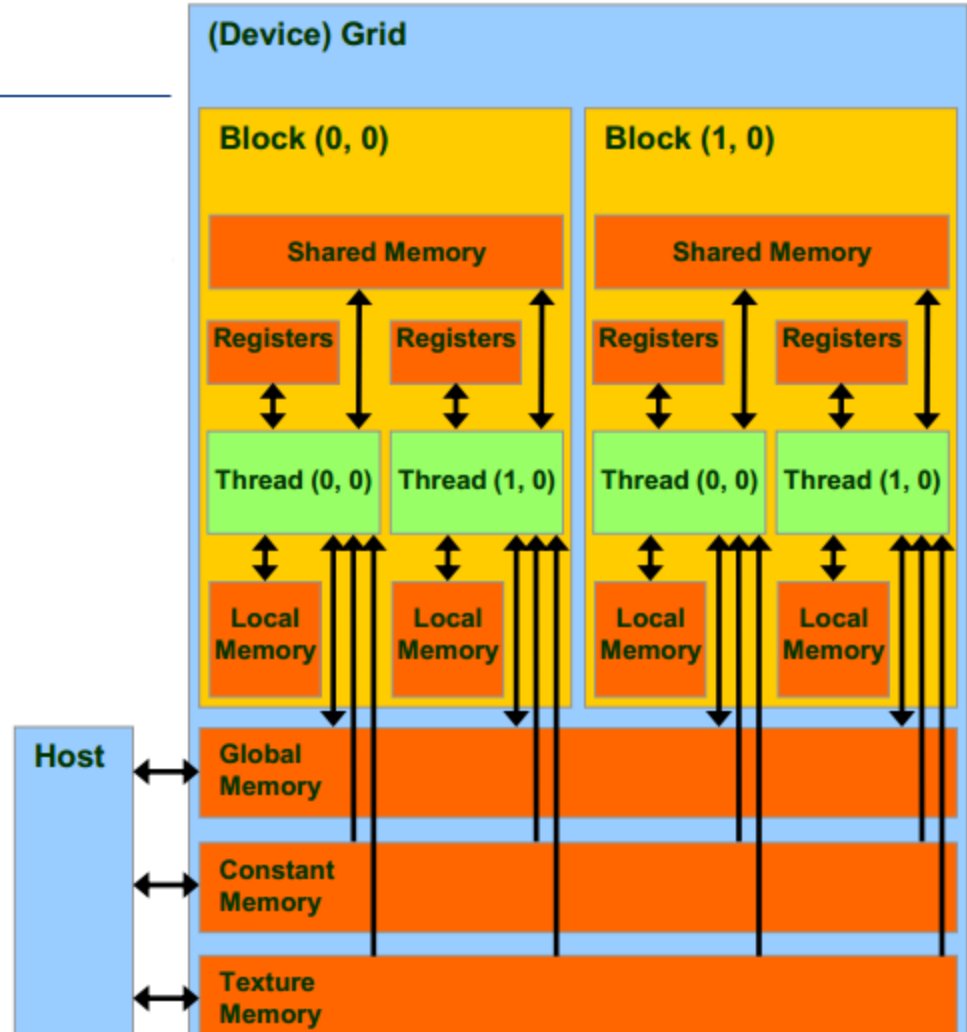


NVIDIA Architecture



NVIDIA Architecture

- Memory model:
 - Local memory for each thread
 - Shared memory
 - Accessible only to the same BLOCK of threads
 - Texture memory (Fast read, special operations)
 - Constant memory
 - Main memory
 - Accessible by all blocks, written from the CPU

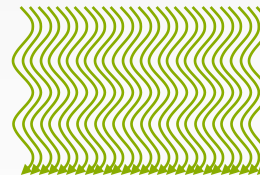


Programming with CUDA

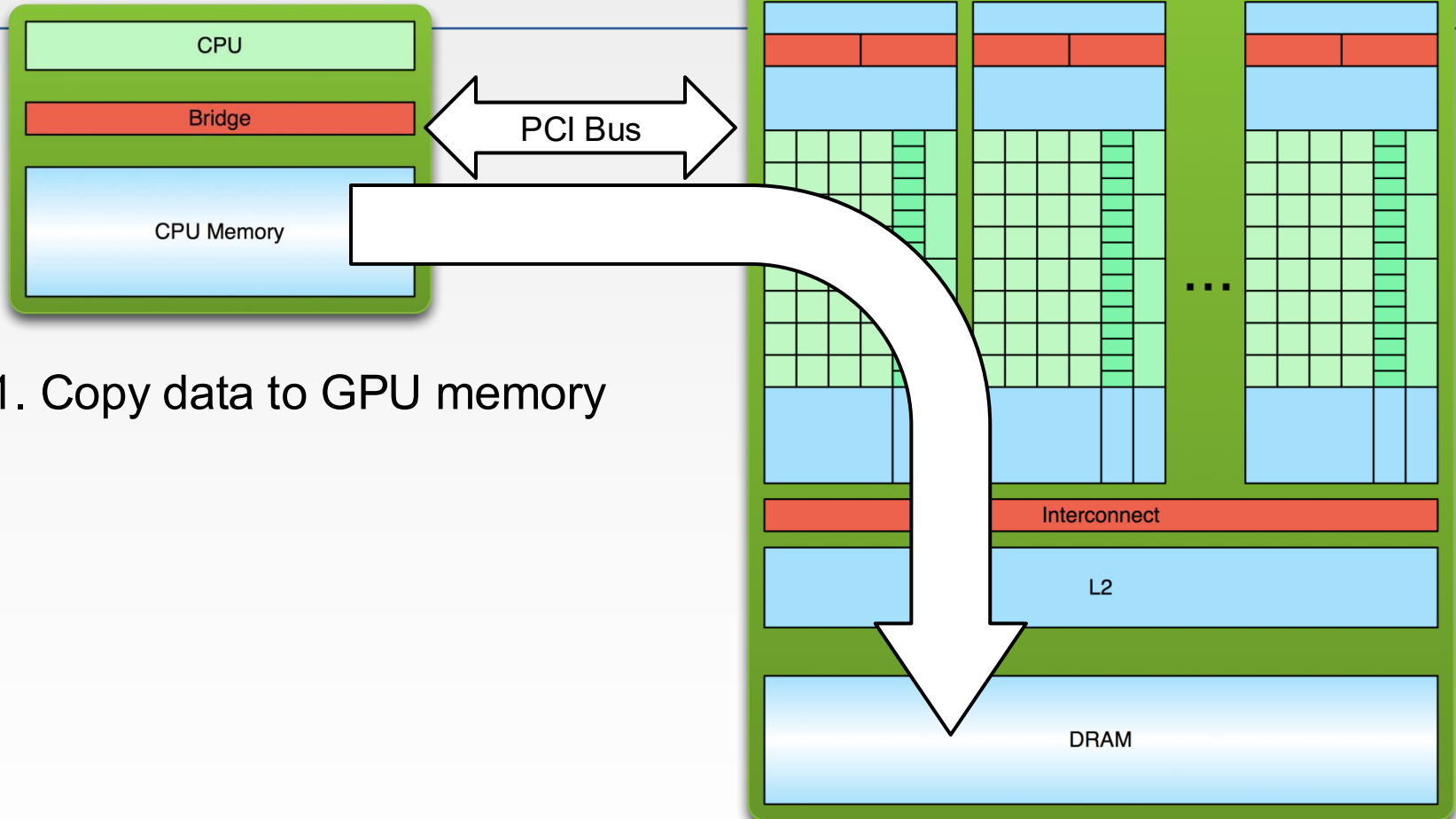
- CUDA is a C extension for programming on massively parallel architectures:
 - Based on libraries and reserved keywords
 - Extends the C language; also supports C++
- Defines sections of code for placing variables and functions:
 - In “host” (CPU memory)
 - In “device” (GPU memory)
- GPU functions are typically called from “host” using predefined methods.

Programming with CUDA

- Our program will contain both types of code:
 - “Host” code
 - Runs sequentially
 - Traditional programming methods
 - Prepares data for the GPU
 - “Device” code
 - “Massively parallel” methods
 - Once the GPU is ready it will create hundreds of threads
 - When finished, it will return control to the CPU, which will retrieve the results

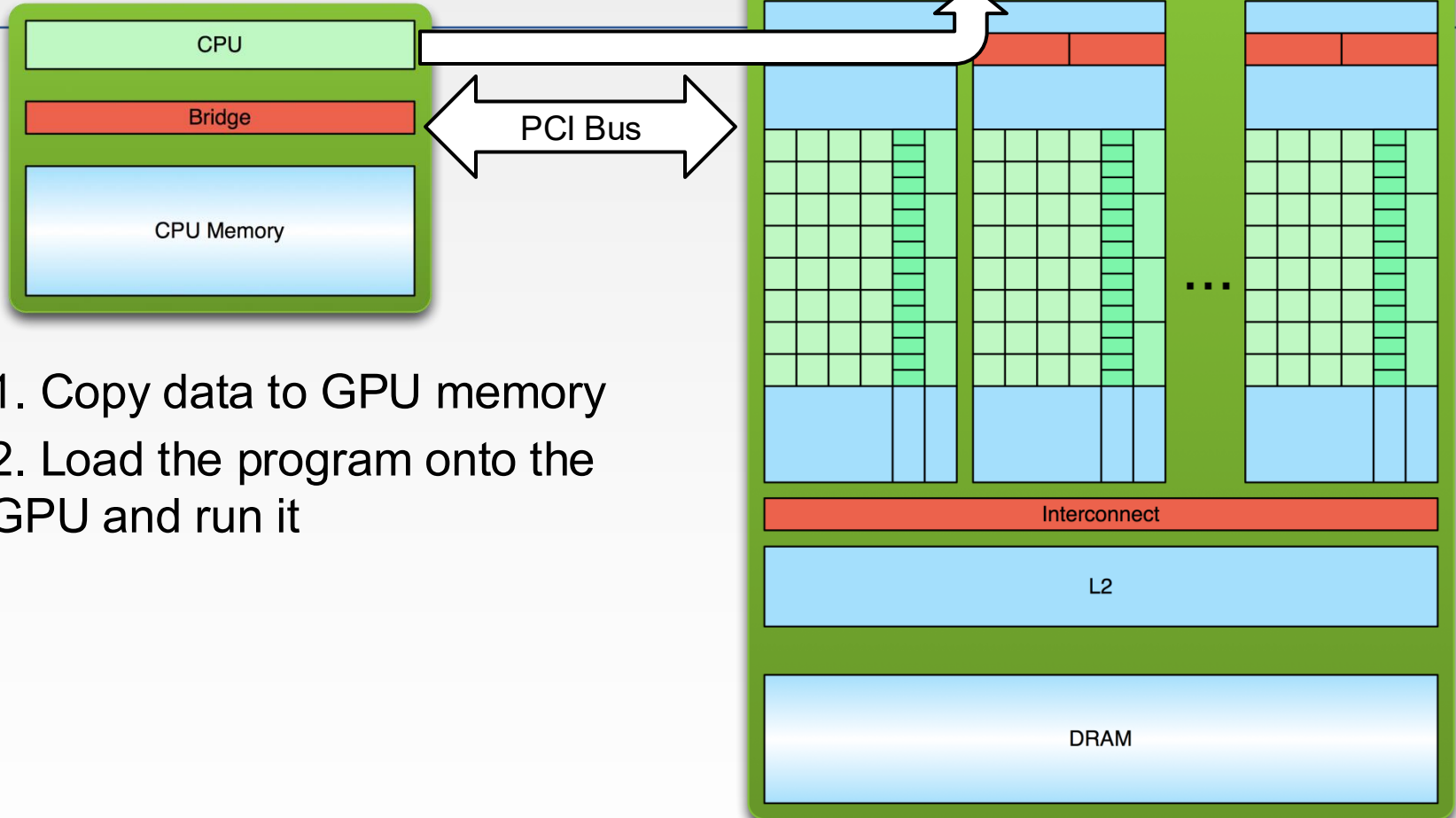


Programming with CUDA



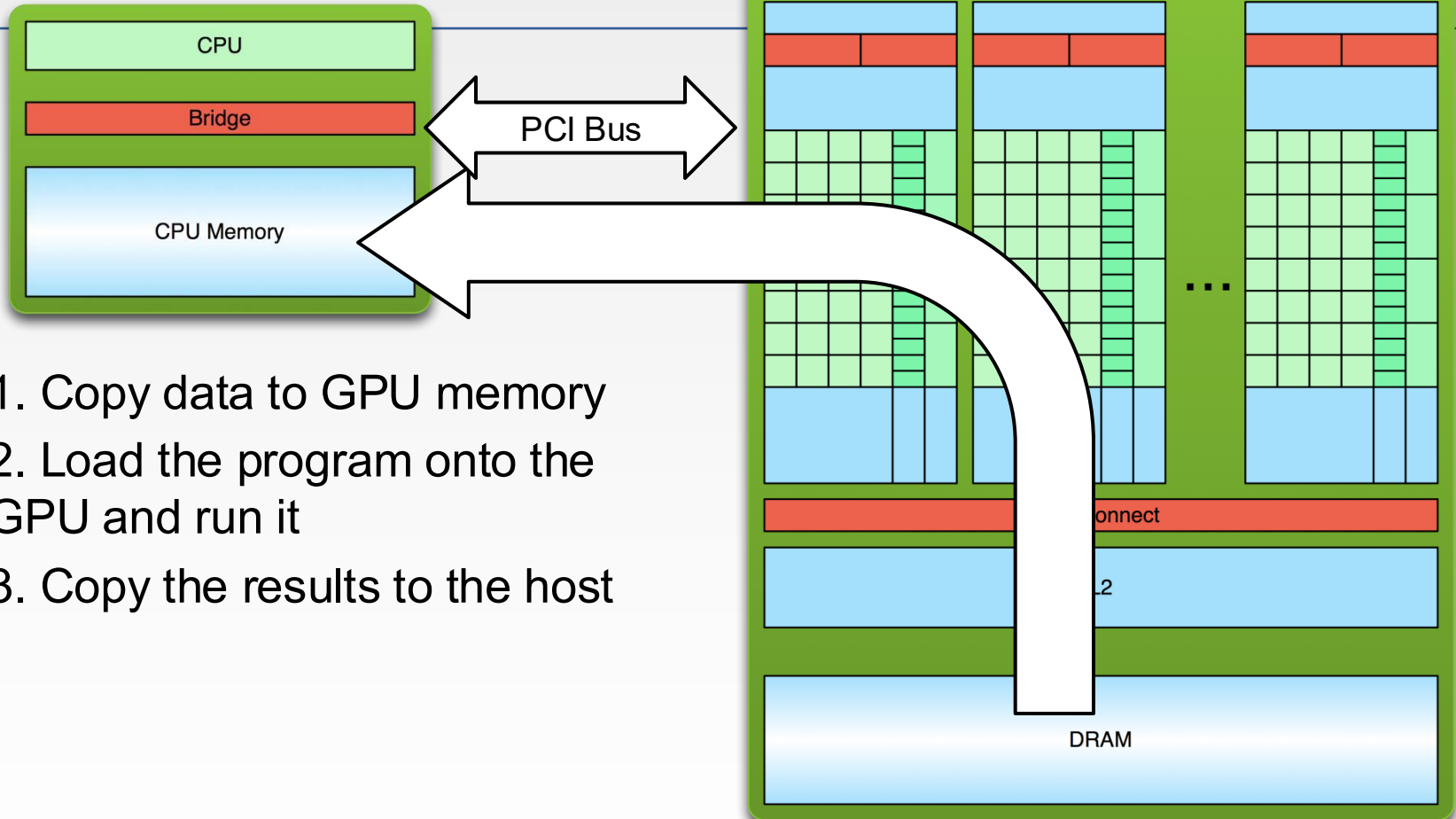
- 1. Copy data to GPU memory

Programming with CUDA



- 1. Copy data to GPU memory
- 2. Load the program onto the GPU and run it

Programming with CUDA



- 1. Copy data to GPU memory
- 2. Load the program onto the GPU and run it
- 3. Copy the results to the host

Programming with CUDA

- “Hello world!”

```
__global__ void helloWorld_k(void) {  
    printf("Hello World!\n");  
}
```

```
int main(void) {  
    HelloWorld_k<<<1,1>>>();  
    return 0;  
}
```

Code visibility

- When defining a CUDA method or kernel, you must specify a visibility type to the compiler. There are three types of visibility:
 - `__global__` :
 - These methods are called from the CPU and act as the 'entry point' to the GPU.
 - `__device__` :
 - These functions are called from a GPU function by another GPU function (another `__global__` or `__device__` method).
 - `__host__` :
 - These methods are called from the CPU and cannot be called from the GPU.

Code visibility

- The visibility types `__device__` and `__host__` can be mixed. The compiler will generate two implementations of the same function, one for each call from the CPU or GPU.

- Ej:

```
__device__ __host__ void helloWorld_k() {  
    printf("Hello World!\n");  
}
```

Kernels

- A kernel is a piece of code that will run on all (or several) of the GPU's processors. The call is specified as follows:

```
int main(void) {  
    int N=1;  
    int M=1;  
    HelloWorld_k<<<N,M>>>();  
    return 0;  
}
```

- “N” blocks, each containing “M” threads, will be created

Parameters

```
__global__ void add_k(int a, int b, int c)
{
    ???

}

int main(void) {
    ...
    add_k<<<1,1>>>(a,b,c) ;
    ...
}
```

Memory allocation

- GPU and CPU memory are located in separate areas; you must specify the allocated memory for each in some way:
 - Device (GPU): Some data can be copied from the CPU via the stack, but CPU memory pointers cannot be dereferenced on the GPU
 - Host (CPU): Data can be copied to the GPU, parameters can be passed via the stack, but GPU pointers (memory allocated on GPU) cannot be dereferenced
- SURE??
 - It depends on the driver, CUDA version, card capabilities, etc...



Memory allocation

- Reservas de memoria en GPU:

- `cudaMalloc()`, `cudaFree()`, `cudaMemcpy()`
- Funciones similares a las de C: `malloc()`, `free()`, `memcpy()`

- Ej:

```
int *d_a, *a;  
cudaMalloc((void **)&d_a, sizeof(int));  
a=(int*)malloc(sizeof(int));  
*a=10;  
cudaMemcpy(d_a,a,sizeof(int,cudaMemcpyHostToDevice);  
cudaMemcpy(a,d_a,sizeof(int,cudaMemcpyDeviceToHost);  
cudaFree(d_a);  
free(a);
```


Thread blocks

```
__global__ void add_k(int *a, int *b, int *c) {
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
}

int main(void) {
    ...
    add_k<<<N,1>>>(a,b,c);
    ...
}
```

Block 0

Block 1

Block 2

Block 3

c[0] = a[0] + b[0];

c[1] = a[1] + b[1];

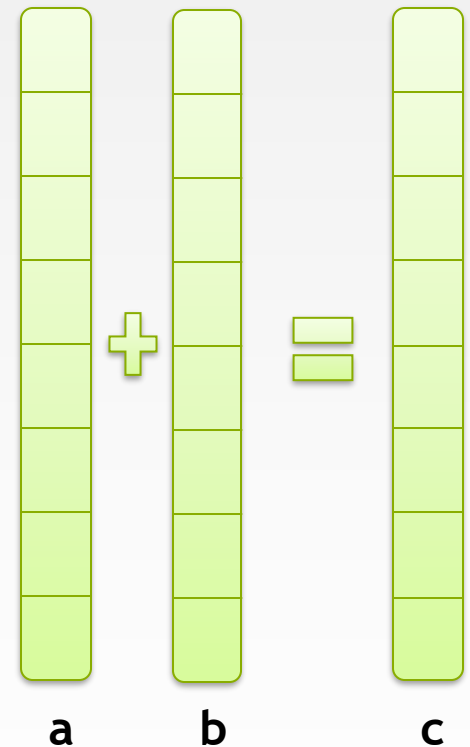
c[2] = a[2] + b[2];

c[3] = a[3] + b[3];

- "N" single-threaded blocks will be created
 - blockIdx indicates the block number in which that piece of code (kernel) is currently running

Thread blocks

- Each kernel accesses its own block identifier; they all access different positions in the array at the same time.
- The threads in each block:
 - Run on a “multiprocessing unit”
 - Share “shared” memory, cache, etc.
 - Can be synchronised
- The optimal approach is to create as many threads as can fit into a block, and to create more blocks than available multiprocessing units.



Thread blocks

```
__global__ void add_k(int *a, int *b, int *c) {  
    c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];  
}  
  
int main(void) {  
    ...  
    add_k<<<1,N>>>(a,b,c);  
    ...  
}
```

- You can also specify how many threads there will be per block. The thread ID within the current block can be accessed via “threadIdx”.

Thread blocks

```
#define MAX_THREADS_BLOCK 128

__global__ void add_k(int *a, int *b, int *c) {
    int index=threadIdx.x+blockIdx.x*MAX_THREADS_BLOCK;
    c[index] = a[index] + b[index];
}

int main(void) {
    ...
    add_k<<<M,MAX_THREADS_BLOCK>>>(a,b,c);
    ...
}
```

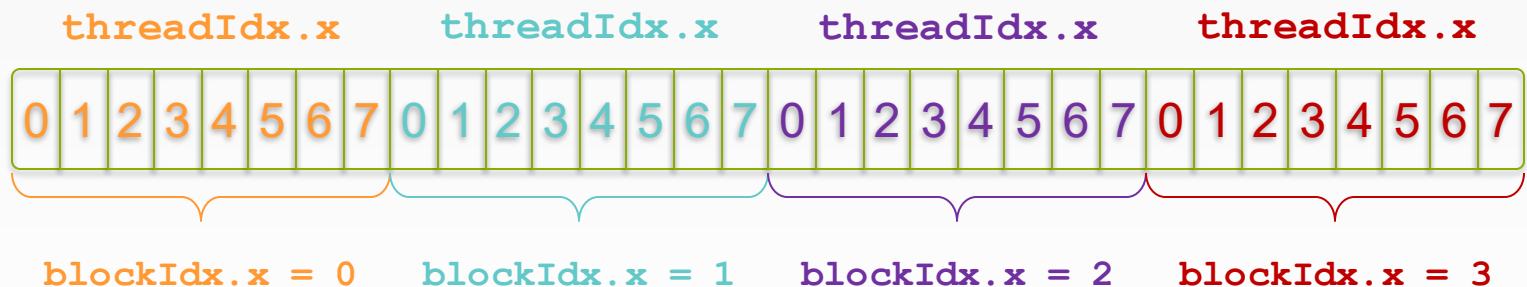
- Finally, it is best to set a maximum block size, depending on the architecture. The kernels will access a position depending on the block and thread they are in.

Thread blocks

- For M threads per block, the index is calculated as follows:

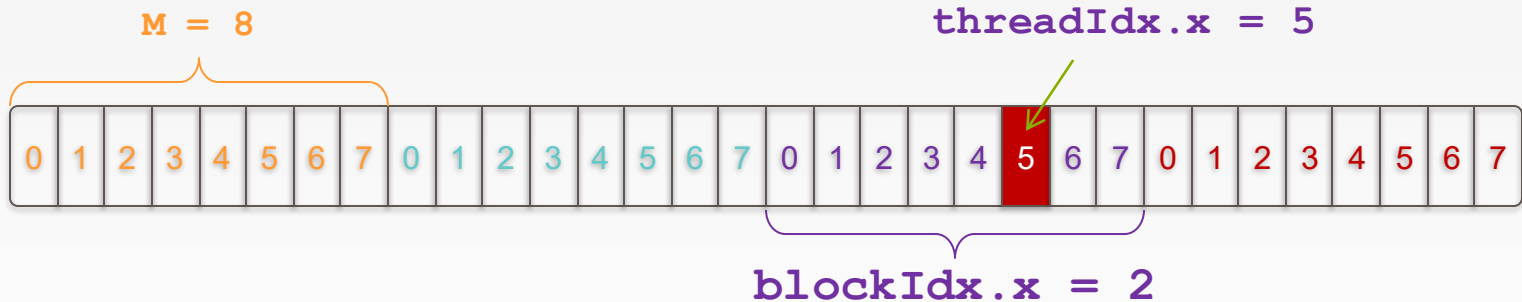
```
int index = threadIdx.x + blockIdx.x * M;
```

- Very similar to indexing a two-dimensional array with “threadIdx” rows by “blockIdx” columns
 - 8 threads/block:



Thread blocks

- Which thread will handle the selected item (21)?



```
int index = threadIdx.x + blockIdx.x * M;
          =          5      +          2      * 8;
          = 21;
```

Thread blocks

- We can use the variable `blockDim.x` to determine at runtime how many threads there are per block

```
__global__ void add_k(int *a, int *b, int *c) {  
    int index = threadIdx.x + blockDim.x * blockDim.x;  
    c[index] = a[index] + b[index];  
}
```

- To automatically calculate the number of blocks and the number of threads per block, with a maximum size of “N”:
- `add<<<(N/THREADS_PER_BLOCK)+1,THREADS_PER_BLOCK>>>(...)`
- The optimal number of threads per block will depend on the architecture and the actual data size (coalescing memory accesses, maximum number of processors, etc.).

Thread blocks

- What happens when the array size is not a multiple of the number of threads per block?

```
__global__ void add_k(int *a, int *b, int *c, int n) {  
    int index = threadIdx.x + blockIdx.x * blockDim.x;  
    if (index < n)  
        c[index] = a[index] + b[index];  
}
```

- To prevent such accesses, the indices are first checked to ensure they are valid; any remaining threads are discarded.

```
add_k<<<((N/M)+1,M)>>>(d_a, d_b, d_c, N);
```


Threads, blocks... why?

- Why deal with two levels of addressing? Why not just use single-threaded blocks?
- We can launch an infinite number of blocks; the scheduler can utilise the entire GPU. However, the threads within a block can
 - Communicate with one another (shared memory)
 - Synchronise for certain tasks
- Blocks are independent processing units (coarse-grained). With threads, we can achieve higher levels of parallelism and faster memory access

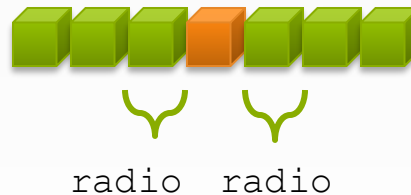
Shared memory

- We can use the shared memory between the threads of a block to speed up memory accesses:
 - Parallel copying from global to local memory
 - Cooperation between all threads in a block
- Example: "linear filter"
 - A linear filter computes an average value for a point/position, based on the surrounding values
 - Example using a one-dimensional (1D) array
$$\text{Arr1} = \{1, 3, 5, 3, 8, 2, 7\}$$
$$\text{Arr1}[i] = (\text{Arr1}[i-1] + \text{Arr1}[i] + \text{Arr1}[i+1]) / 3$$

Shared memory

- Each thread calculates one element, but it needs to access three elements in main memory

```
__global__ void filter_k(int *orig, int *result, int n) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    if (index < n)
        result[index] = orig[index-1] + orig[index] +
                        orig[index+1];
    result[index] = result[index]/3;
}
```



Shared memory

- In these cases, the cache is not sufficient; we can use shared memory to take advantage of parallel reads by the threads:
 - Threads can share data via shared memory
 - User-configurable memory, very fast
- It is declared using the `__shared__` directive; that amount is reserved for each block.
- NOTE: This data is not visible to other blocks; in some cases, it may be necessary to copy-back the data

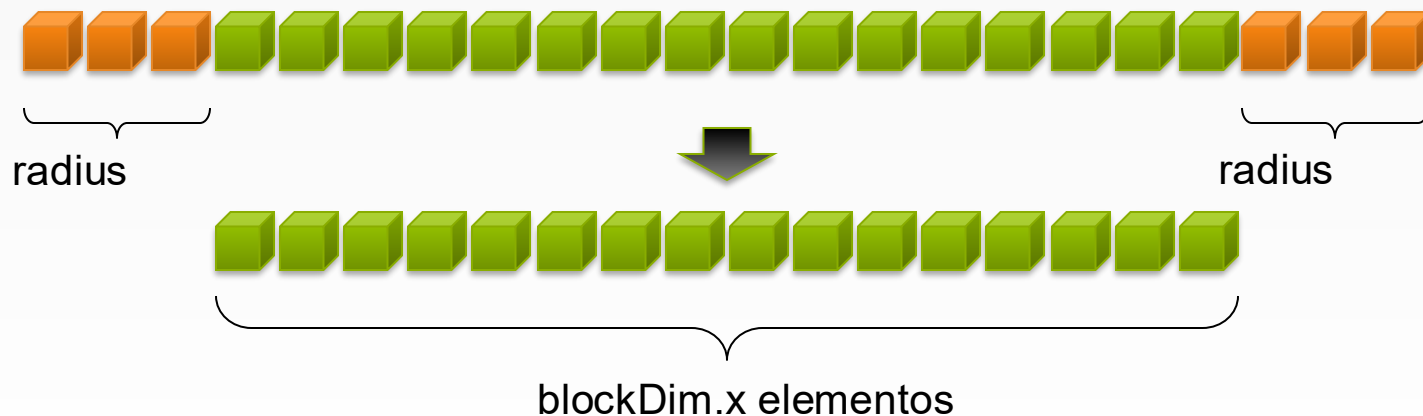
Shared memory

- Since the linear filter uses data within a radius of one unit, for every N threads in a block we will need N+2 data points:

Array[N];

NBLOCKS = N/THREADS_PER_BLOCK

SHARED_SIZE=THREADS_PER_BLOCK+2*RADIUS



Shared memory

- `void __syncthreads();`
- It synchronises the threads belonging to a block:
 - Prevents RAW / WAR / WAW risks
- All threads must reach the barrier before reading from shared memory
 - Read/write conditions must be the same for all threads

Shared memory

```
__global__ void filterSM_k(int *in, int *out) {
    // allocate shared memory
    __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
    int g_index = threadIdx.x + blockIdx.x * blockDim.x;
    int l_index = threadIdx.x + RADIUS;
```



```
// read the selected item, store in shared memory
```

```
temp[l_index] = in[g_index]/3; //optimización
```

```
if (threadIdx.x < RADIUS) { //first thread
```

```
    //read left "radius", RADIUS=1
```

```
    temp[l_index - RADIUS] = in[g_index - RADIUS]/3;
```

```
    //read right "radius" , RADIUS=1
```

```
    temp[l_index + BLOCK_SIZE] = in[g_index + BLOCK_SIZE]/3;
```

```
}
```



```
//SYNCHRONIZE!!
```

```
__syncthreads()
```

Shared memory

```
// apply filter
int result = 0;
for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
    result += temp[l_index + offset]; //accedemos a compartida

// store results
out[g_index] = result;
}
```

....

//kernel call from CPU

MSHARED_SIZE=M+2*RADIUS;

FilterSM_k<<<((N/M)+1,MSHARED_SIZE,>>>(in,out) ;

....

- In this way, we read the data in parallel, save on main memory bandwidth, and perform coalescent read/write operations